

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
— o0o —



Báo cáo cá nhân:
UC004 - PlanImportAllocation

Hệ thống đặt hàng nhập khẩu

Học phần: Phát triển phần mềm theo chuẩn ITSS
Mã học phần: IT4549
Nhóm 15

Sinh viên thực hiện:

Phan Khánh Vũ - 20235880

Lớp: ITSS

Giảng viên hướng dẫn: TS. Trịnh Tuấn Đạt

Hà Nội - 2026

Mục lục

1	Tài liệu đặc tả yêu cầu phần mềm SRS	3
1.1	Giới thiệu phạm vi nghiệp vụ cá nhân	3
1.2	Phân công nhiệm vụ nhóm	3
1.3	Đặc tả chi tiết Use Case UC004 – PlanImportAllocation	3
1.4	Sơ đồ đặc tả	5
2	Tài liệu phân tích Use Case	7
2.1	Mô tả luồng xử lý phân tích	7
2.2	Sơ đồ lớp mức phân tích	8
3	Thiết kế giao diện và Subsystem	9
3.1	Thiết kế và đặc tả màn hình	9
3.1.1	Đặc tả các thành phần giao diện màn hình AllocationView . . .	9
3.1.2	Giao diện chương trình minh họa	10
3.2	Thiết kế Subsystem	11
4	Thiết kế chi tiết phần mềm	12
4.1	Thiết kế động và tĩnh chi tiết	12
4.2	Biểu đồ phụ thuộc gói	14
5	Nguyên lý thiết kế và mẫu thiết kế áp dụng	15
5.1	Nguyên lý thiết kế	15
5.2	Mẫu thiết kế	15
6	Kiểm thử phần mềm	16
6.1	Thiết kế các ca kiểm thử	16
6.1.1	Thiết kế kiểm thử hộp đen (Black-box Testing)	16
6.1.2	Thiết kế kiểm thử hộp trắng (White-box Testing - Độ bao phủ C1) .	16
6.2	Mã nguồn kiểm thử tự động	17

Chương 1

Tài liệu đặc tả yêu cầu phần mềm SRS

1.1 Giới thiệu phạm vi nghiệp vụ cá nhân

Phạm vi công việc cá nhân tập trung vào thuật toán phân bổ phương án đặt hàng cho từng mặt hàng dựa trên dữ liệu tồn kho và thời gian vận chuyển của các Site, thực hiện việc ưu tiên phương tiện vận chuyển và chọn lọc số lượng tối ưu.

1.2 Phân công nhiệm vụ nhóm

Dưới đây là bảng phân công vai trò phụ trách use case của các thành viên trong nhóm:

Bảng 1.1:

Thành viên	MSSV	Use case phụ trách
Nguyễn Hữu Nhân	20235798	UC006 - ReceiveAndReconcileGoods
Vũ Đức Hoàng Anh	20235658	UC001 - ManageImportRequests, UC007 - ManageSystem
Trần Đăng Sinh	20235821	UC003 - CollectInventoryData
Phan Khánh Vũ	20235880	UC004 - PlanImportAllocation
Lương Quốc Khánh	20235754	UC005 - PlaceOverseasOrders
Nguyễn Quý Đức	20235682	UC002 - ManageSiteInformation

1.3 Đặc tả chi tiết Use Case UC004 – PlanImportAllocation

Bảng 1.2: Đặc tả Use Case UC-004: PlanImportAllocation

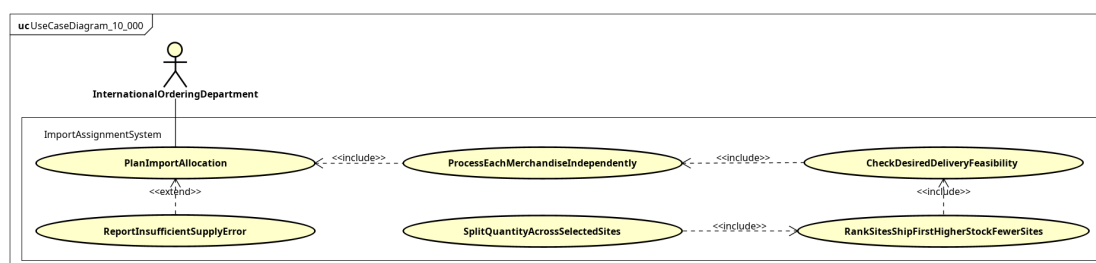
Đặc tả Use Case			
Mã UseCase	UC-004		
Tên UseCase	PlanImportAllocation		
Tác nhân	InternationalOrderingDepartment		
Tiền điều kiện	Đã có dữ liệu yêu cầu nhập và dữ liệu tồn kho theo Site; thông tin thời gian vận chuyển ship/air của Site đã cập nhật.		
Luồng sự kiện chính	STT	Thực hiện bởi	Hành động
	1	InternationalOrderingDepartment	Khởi tạo lập kế hoạch phân bổ cho yêu cầu nhập hiện tại.
	2	System	Xử lý từng mặt hàng một cách độc lập.
	3	System	Kiểm tra Site có thể đáp ứng ngày nhận mong muốn theo thời gian vận chuyển.
	4	System	Xếp hạng Site theo ưu tiên: ship trước, tồn kho lớn hơn, số Site ít hơn.
	5	System	Phân bổ số lượng đặt cho một hoặc nhiều Site đến khi đáp ứng đủ nhu cầu.
	6	InternationalOrderingDepartment	Xem và xác nhận phương án phân bổ nhập hàng.
Luồng sự kiện thay thế	STT	Thực hiện bởi	Hành động
	3a	System	Không có Site nào đáp ứng ngày nhận mong muốn, hệ thống đánh dấu mặt hàng rủi ro trễ hạn.
	5a	System	Tổng tồn kho khả dụng nhỏ hơn nhu cầu, hệ thống phát sinh lỗi thiếu nguồn cung.
	6a	InternationalOrderingDepartment	Không chấp nhận phương án, điều chỉnh tham số ưu tiên và chạy lại phân bổ.
Hậu điều kiện	Phương án phân bổ được lưu theo từng mặt hàng và từng Site, kèm phương thức vận chuyển dự kiến; các lỗi thiếu nguồn cung được ghi nhận nếu có.		

* Dữ liệu đầu vào của usecase PlanImportAllocation gồm các trường dữ liệu sau:

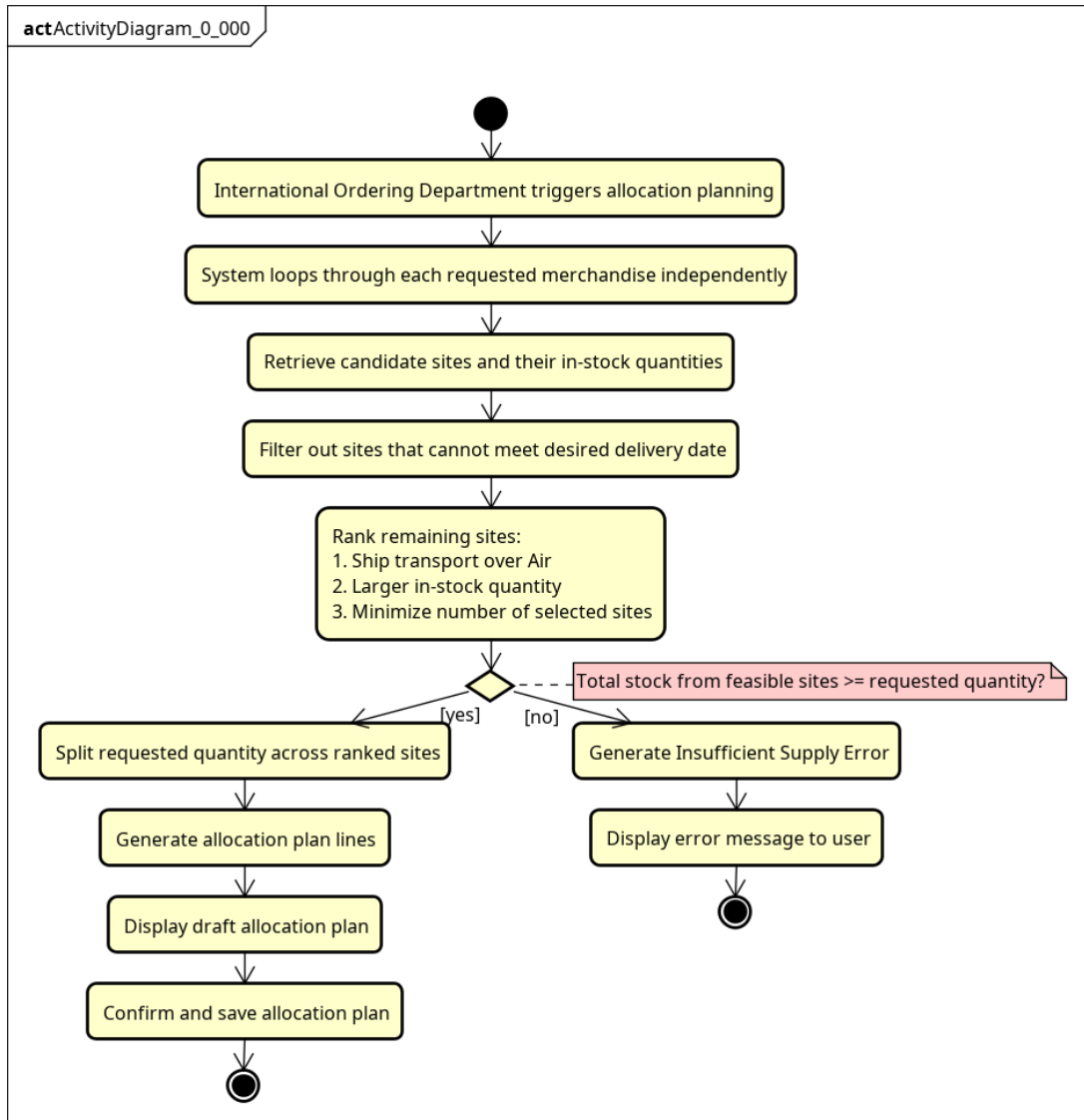
Bảng 1.3: Dữ liệu đầu vào UC-004

STT	Trường dữ liệu	Mô tả	Bắt buộc	Điều kiện hợp lệ	Ví dụ
1	desiredDeliveryDate	Ngày nhận mong muốn của từng mặt hàng	Có	Là ngày hợp lệ	2026-06-01
2	availableSites	Danh sách Site có hàng cho mặt hàng	Có	Mỗi Site có siteCode hợp lệ và tồn kho đi kèm	SITE-SEA-01, SITE-EU-04
3	inStockQuantityBySite	Số lượng tồn kho theo từng Site	Có	Số nguyên không âm	320, 150
4	deliveryDaysByShipOrAir	Số ngày vận chuyển theo từng phương thức của Site	Có	Số nguyên dương	ship=18, air=5

1.4 Sơ đồ đặc tả



Hình 1.1: Use case phân rã UC004 - PlanImportAllocation



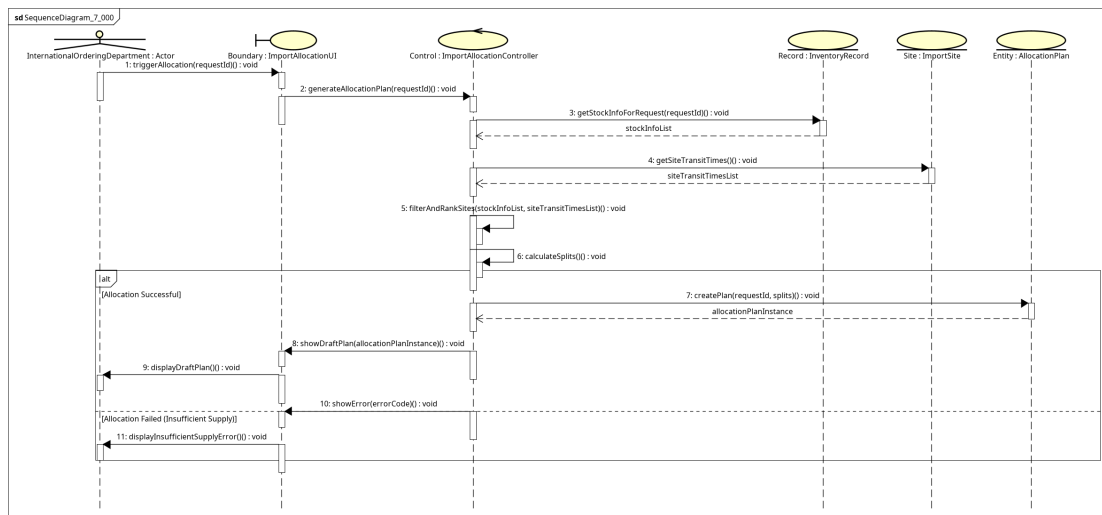
Hình 1.2: Activity diagram UC004 - PlanImportAllocation

Chương 2

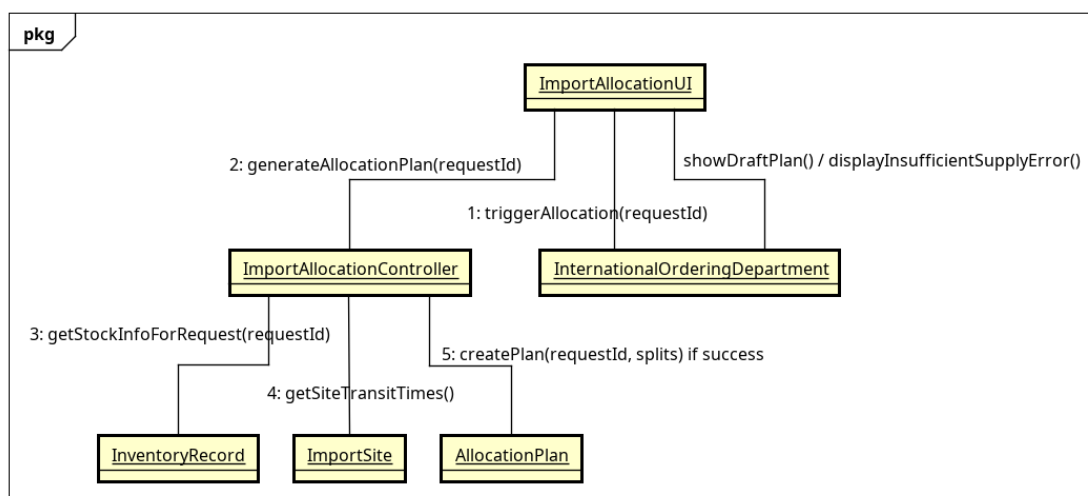
Tài liệu phân tích Use Case

2.1 Mô tả luồng xử lý phân tích

Phân tích tương tác giữa biên, điều khiển và các lớp thực thể để tính toán phương án phân bổ.

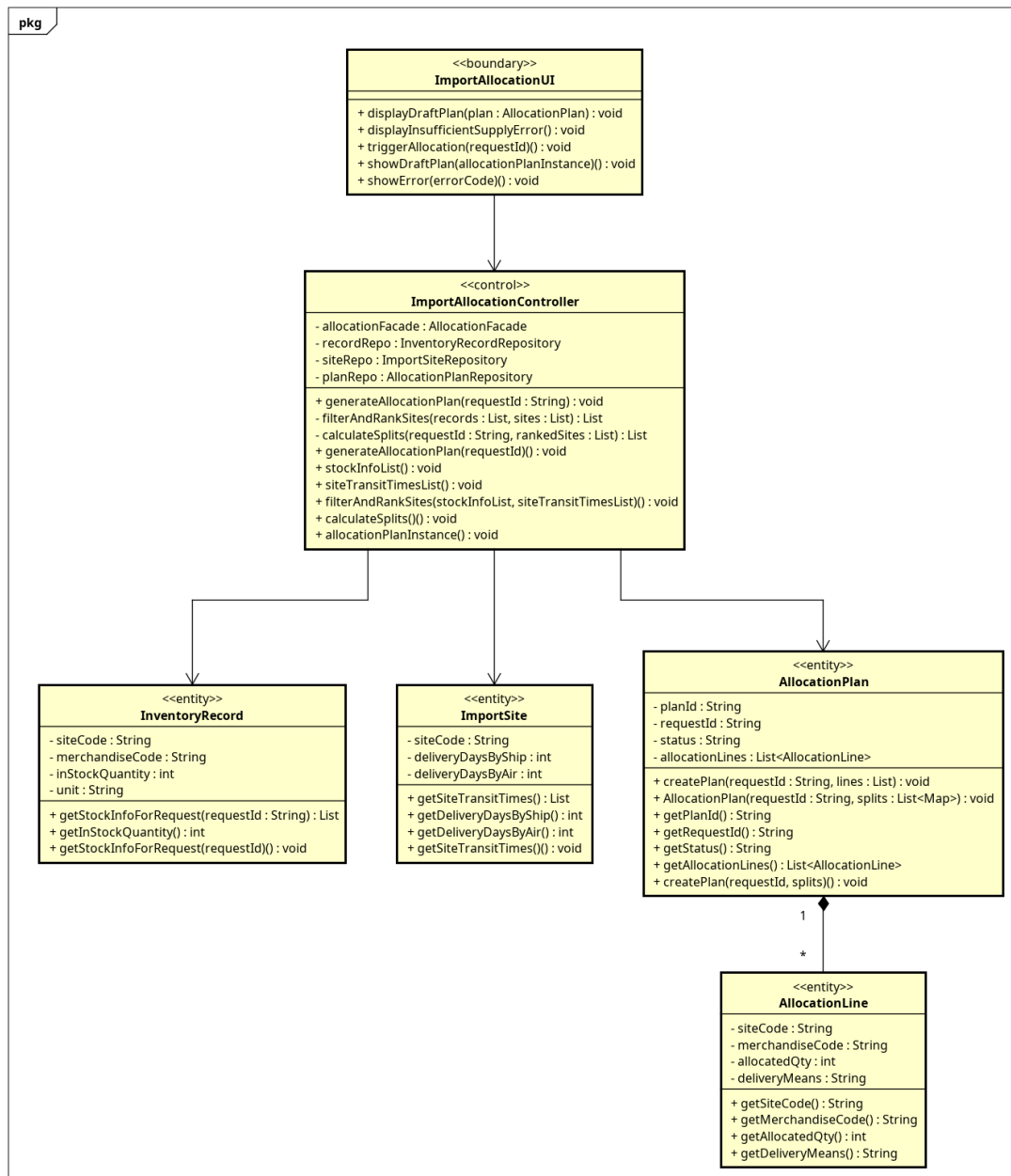


Hình 2.1: Sequence diagram mức phân tích UC004 - PlanImportAllocation



Hình 2.2: Communication diagram mức phân tích UC004 - PlanImportAllocation

2.2 Sơ đồ lớp mức phân tích



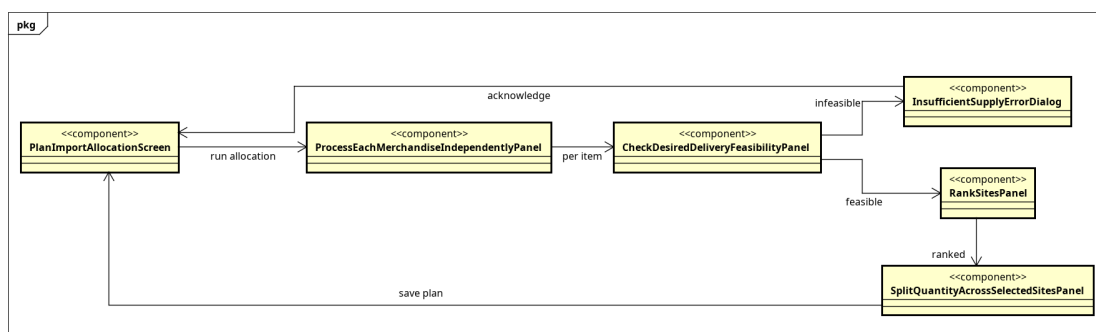
Hình 2.3: Class diagram mức phân tích UC004 - PlanImportAllocation

Chương 3

Thiết kế giao diện và Subsystem

3.1 Thiết kế và đặc tả màn hình

Màn hình AllocationView hỗ trợ việc sinh phương án phân bổ tự động, cảnh báo thiếu hụt nguồn cung và cho phép điều chỉnh thủ công trước khi xác nhận.



Hình 3.1: Screen transition diagram UC004 - PlanImportAllocation

3.1.1 Đặc tả các thành phần giao diện màn hình Allocation-View

Bảng 3.1:

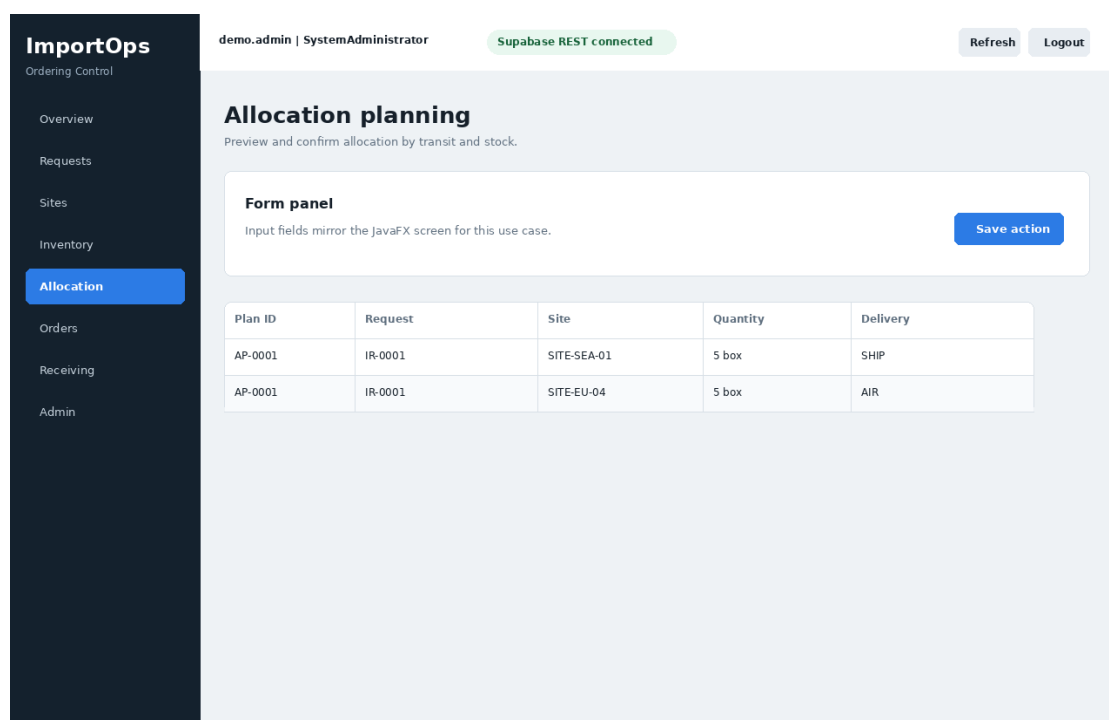
STT	Tên thành phần	Phân loại	Giải thích chi tiết
1	Tiêu đề Allocation	Label	Hiển thị tên màn hình lập phương án phân bổ.
2	Ô requestId	Combo Box	Chọn request đã có dữ liệu tồn kho để lập phương án.
3	Bảng InventoryRecord	Data Grid/Table	Hiển thị tồn kho và shipping time của các Site ứng viên.
4	Bảng Allocation preview	Data Grid/Table	Hiển thị siteCode, merchandise Code, quantity và deliveryMeans dự kiến.

(tiếp tục ở trang sau)

Bảng 3.1: (tiếp)

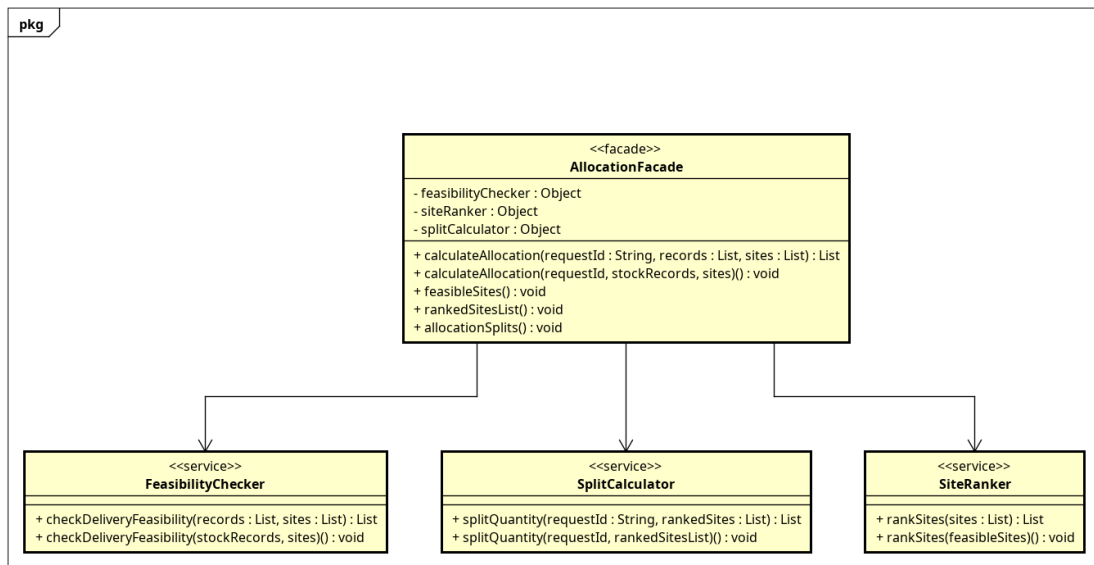
STT	Tên thành phần	Phân loại	Giải thích chi tiết
5	Cột <code>deliveryMeans</code>	Table Column	Cho biết phương thức vận chuyển ship delivery hoặc air delivery.
6	Cột <code>shortageWarning</code>	Status Column	Cảnh báo khi không đủ nguồn cung đáp ứng ngày nhận mong muốn.
7	Nút Generate plan	Button	Tính phương án phân bổ theo các tiêu chí ưu tiên.
8	Nút Manual adjust	Button	Cho phép điều chỉnh số lượng phân bổ trước khi xác nhận.
9	Nút Confirm plan	Button	Xác nhận <code>AllocationPlan</code> và chuyển request sang <code>Allocated</code> .
10	Vùng thông báo	Message Label	Hiển thị lỗi thiếu nguồn cung hoặc kết quả xác nhận.

3.1.2 Giao diện chương trình minh họa

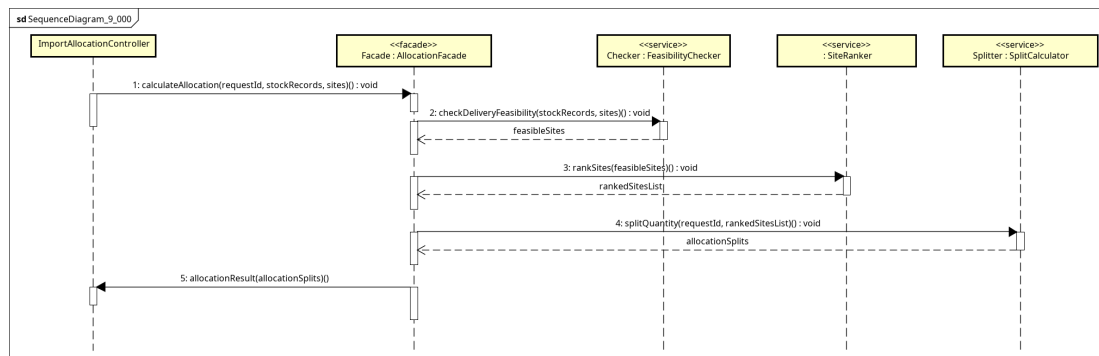


Hình 3.2: Giao diện lập phương án phân bổ đặt hàng

3.2 Thiết kế Subsystem



Hình 3.3: Class diagram subsystem UC004 - PlanImportAllocation

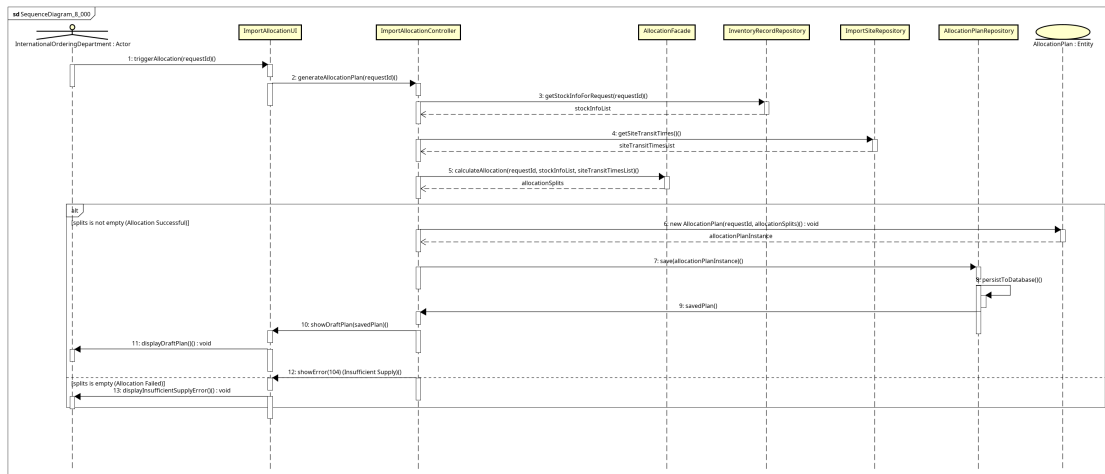


Hình 3.4: Sequence diagram subsystem UC004 - PlanImportAllocation

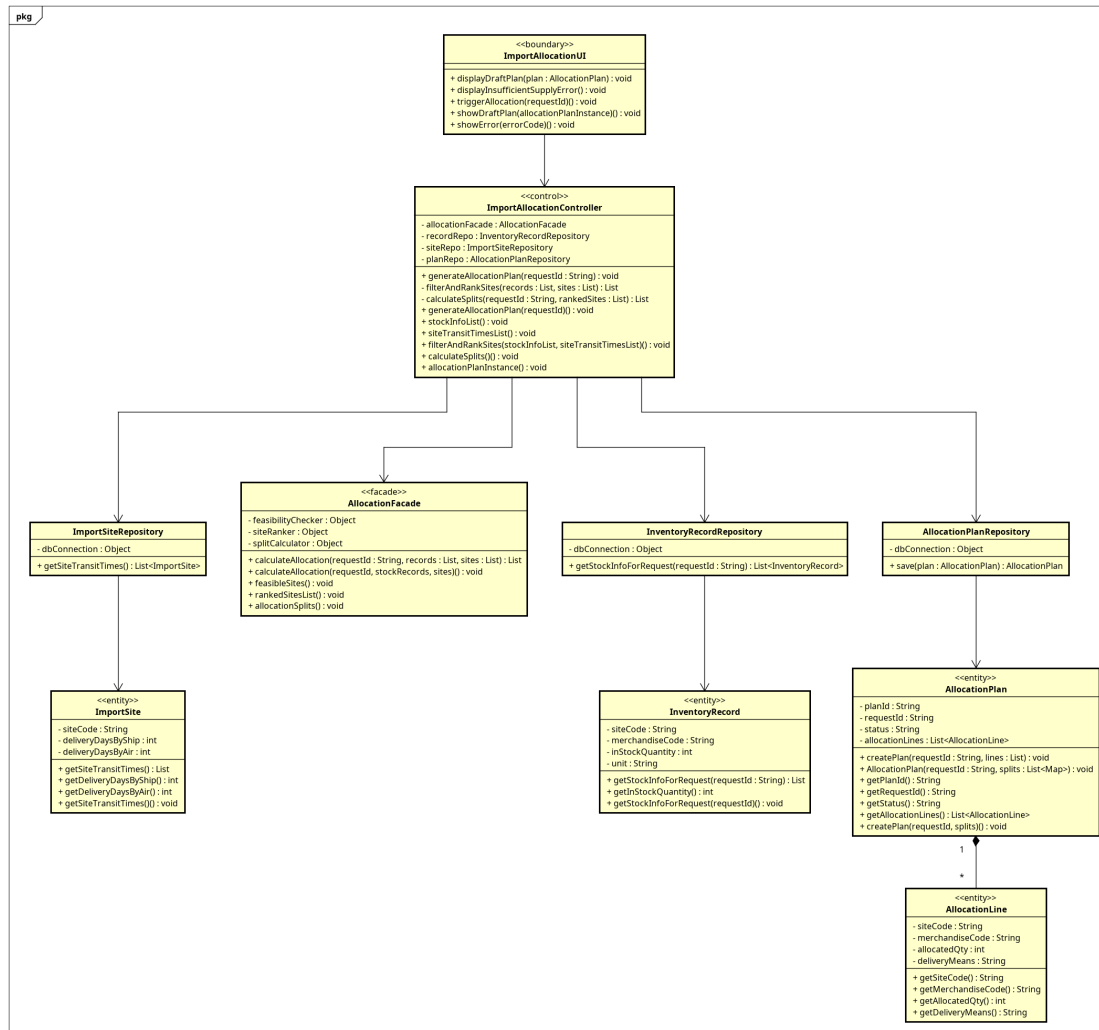
Chương 4

Thiết kế chi tiết phần mềm

4.1 Thiết kế động và tĩnh chi tiết

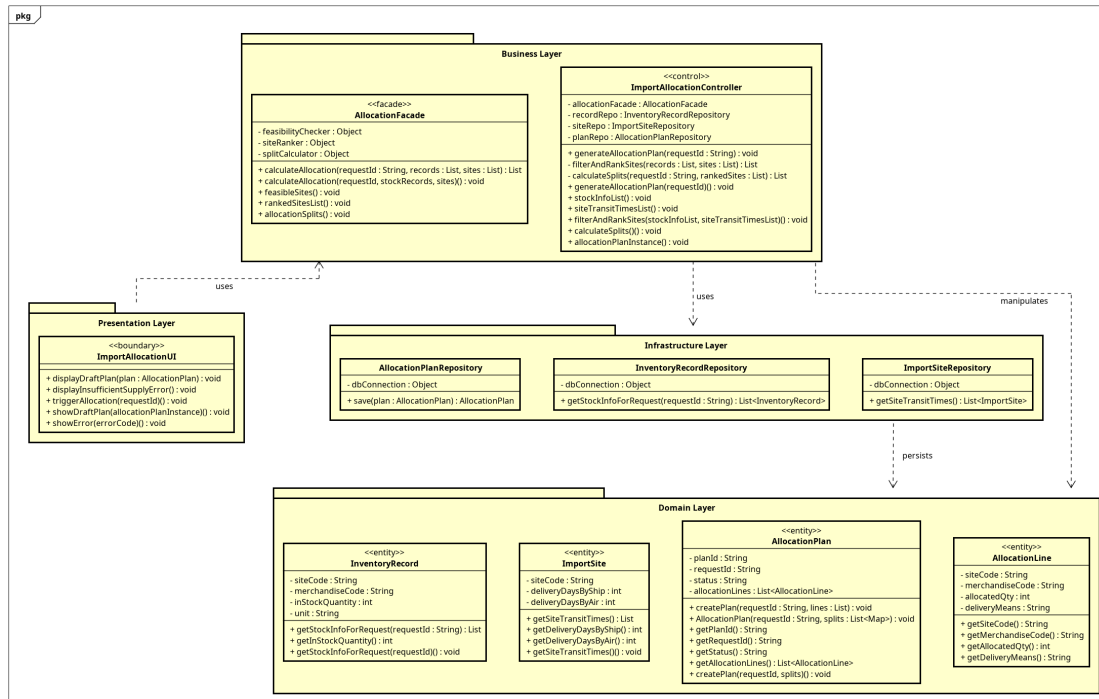


Hình 4.1: Sequence diagram mức thiết kế UC004 - PlanImportAllocation



Hình 4.2: Class diagram mức thiết kế UC004 - PlanImportAllocation

4.2 Biểu đồ phụ thuộc gói



Hình 4.3: Package dependency diagram UC004 - PlanImportAllocation

Chương 5

Nguyên lý thiết kế và mẫu thiết kế áp dụng

5.1 Nguyên lý thiết kế

- **Single Responsibility Principle (SRP):** Tách biệt hoàn toàn thuật toán phân bổ ra khỏi giao diện điều phối. Toàn bộ logic gom cụm, xếp hạng và chọn phương thức vận chuyển nằm trong lớp dịch vụ ứng dụng.
- **High Cohesion:** Các thành phần trong engine tính toán phân bổ được thiết kế tập trung, hỗ trợ chặt chẽ lẫn nhau (FeasibilityChecker kiểm tra thời gian vận chuyển, SiteRanker xếp hạng độ ưu tiên Site).

5.2 Mẫu thiết kế

- **Facade:** Lớp facade tập trung điều hướng yêu cầu tính toán từ UI sang lớp xử lý phân bổ.
- **Strategy Pattern:** Sử dụng chiến lược phân bổ linh hoạt. Tiêu chí ưu tiên vận chuyển bằng tàu thủy (SHIP) trước hàng không (AIR), tiếp đến ưu tiên tồn kho lớn có thể được đóng gói dưới dạng các chính sách phân bổ, giúp thuật toán dễ thay đổi trong tương lai mà không ảnh hưởng tới lớp gọi.

Chương 6

Kiểm thử phần mềm

6.1 Thiết kế các ca kiểm thử

6.1.1 Thiết kế kiểm thử hộp đen (Black-box Testing)

Thiết kế các ca kiểm thử cho phương thức tính toán phân bổ ‘plan’ trong lớp dịch vụ ‘AllocationService’:

Bảng 6.1:

Ca kiểm thử	Dữ liệu đầu vào	Kết quả mong đợi	Nhóm phân vùng
TC-UC004-BB1	Request có đủ nguồn cung tại các Site đáp ứng ngày nhận mong muốn.	Phân bổ thành công, tạo phương án và chuyển trạng thái request thành Allocated.	Đầy đủ nguồn cung.
TC-UC004-BB2	Site có cả đường biển (đáp ứng ngày mong muốn) và hàng không.	Ưu tiên chọn hình thức vận chuyển bằng tàu thủy (SHIP) trước hàng không (AIR).	Ưu tiên phương thức vận chuyển.
TC-UC004-BB3	Tổng tồn kho đáp ứng của các Site nhỏ hơn số lượng yêu cầu đặt hàng.	Hệ thống ném lỗi BusinessException cảnh báo thiếu nguồn cung.	Thiếu hụt nguồn cung nghiêm trọng.

6.1.2 Thiết kế kiểm thử hộp trắng (White-box Testing - Độ bao phủ C1)

Bao phủ các nhánh kiểm tra và gom lượng phân bổ:

- **Nhánh 1:** Tổng lượng tồn kho khả dụng từ các Site lớn hơn hoặc bằng số lượng yêu cầu → Lập phương án phân bổ và đánh dấu request là ‘ALLOCATED’.

- **Nhánh 2:** Tổng lượng tồn kho khả dụng nhỏ hơn số lượng yêu cầu → Ném lỗi ‘BusinessException’ dùng xử lý.
- **Nhánh 3:** Site đáp ứng bằng đường biển → Chọn SHIP.
- **Nhánh 4:** Site chỉ đáp ứng bằng đường hàng không → Chọn AIR.

6.2 Mã nguồn kiểm thử tự động

Mã nguồn chương trình kiểm thử tự động bằng JUnit 5:

```

1 package vn.edu.hust.itss.group15.usecases;
2
3 import static org.junit.jupiter.api.Assertions.assertEquals;
4 import static org.junit.jupiter.api.Assertions.assertThrows;
5
6 import java.time.LocalDate;
7 import java.util.List;
8 import org.junit.jupiter.api.Test;
9 import vn.edu.hust.itss.group15.application.BusinessException;
10 import vn.edu.hust.itss.group15.application.
    ImportOrderingFacade;
11 import vn.edu.hust.itss.group15.domain.AllocationPlan;
12 import vn.edu.hust.itss.group15.domain.DeliveryMeans;
13 import vn.edu.hust.itss.group15.domain.ImportRequest;
14 import vn.edu.hust.itss.group15.domain.ImportRequestItem;
15 import vn.edu.hust.itss.group15.domain.RequestStatus;
16 import vn.edu.hust.itss.group15.infrastructure.persistence.
    memory.InMemoryStore;
17
18 class Uc004AllocationTest {
19     @Test
20     void
        tcUc00401PlansAllocationByShipBeforeAirAndMarksRequestAllocated
        () {
21         ImportOrderingFacade facade = facade();
22         ImportRequest request = facade.importRequests().create(List
            .of(item("MH-001", 10, 25)));
23         facade.inventory().recordStock("SITE-SEA-01", "MH-001", 5,
            "box");
24         facade.inventory().recordStock("SITE-EU-04", "MH-001", 20,
            "box");
25
26         AllocationPlan plan = facade.allocation().plan(request.
            requestId());
27
28         assertEquals(2, plan.lines().size());
29         assertEquals("SITE-SEA-01", plan.lines().get(0).siteCode())
            ;
30         assertEquals(DeliveryMeans.SHIP, plan.lines().get(0).
            deliveryMeans());

```

```

31     assertEquals("SITE-EU-04", plan.lines().get(1).siteCode());
32     assertEquals(DeliveryMeans.AIR, plan.lines().get(1).
33         deliveryMeans());
34     assertEquals(RequestStatus.ALLOCATED, facade.importRequests
35         ().find(request.requestId()).status());
36 }
37
38 @Test
39 void tcUc00402FailsAllocationWhenSupplyIsInsufficient() {
40     ImportOrderingFacade facade = facade();
41     ImportRequest request = facade.importRequests().create(List
42         .of(item("MH-004", 10, 40)));
43     facade.inventory().recordStock("SITE-AIR-02", "MH-004", 5,
44         "box");
45
46     assertThrows(BusinessException.class, () -> facade.
47         allocation().plan(request.requestId()));
48 }
49
50 private ImportOrderingFacade facade() {
51     return new ImportOrderingFacade(new InMemoryStore());
52 }
53
54 private ImportRequestItem item(String merchandiseCode, int
55     quantity, int daysFromNow) {
56     return new ImportRequestItem(merchandiseCode, quantity, "
57         box", LocalDate.now().plusDays(daysFromNow));
58 }
59 }

```